

Pooja (handwritten)
024250112

Test

Roll No. :- 41
Sect = B

Ques) Node.js executes all synchronous code first (A & D)
Next, microtask (like Promise.resolve().then())
are executed immediately after the current call
stack clears printing C. and then print B.

Output $\rightarrow$ A

D
B
C

2) ^{calc} function add(a,b) {
return a+b;

}
function subtract(a,b) {
return a-b;

}
module.exports = { add, subtract };

^{calc}
const { add, subtract } = require('./calc');
console.log(add(4,5));

3) const express = require('express');
const app = express();
app.use(express.json());

// c) Custom authentication

const requireAuth = (req, res, next) $\Rightarrow$ {

```

const apiKey = req.headers['x-api-key'];
if (apiKey === 'secret123') {
  next();
} else {
  res.status(401).json({ error: 'Unauthorized' });
}

```

```

// 3 } Public get route
app.get('/api/books', (req, res) => {
  res.json({ message: 'Public book list' });
});
app.listen(3000);

```

```

4 } const express = require('express');
const app = express();
app.use(express.json());
let books = [{ id: '101', title: 'drama', author: 'Shen' }];
app.get('/api/books', (req, res) => {
  res.status(200).json(books);
});
app.post('/api/books', (req, res) => {
  const { title, author } = req.body;
  const newBook = {
    id: books.length ? books[books.length - 1].id + 1 :
    title,
    author
  };

```

```

books.push(new Book);
res.status(201).json(new Book);
});
app.delete('/api/books/:id', (req, res) => {
  const bookId = parseInt(req.params.id, 10);
  const index = books.findIndex(b => b.id === bookId);
  if (index === -1) {
    return res.status(404).json({ error: 'Book not found' });
  }
  books.splice(index, 1);
  res.status(202).json({ message: 'Book deleted' });
});
app.listen(3000);
5) app.get('/api/books/:id', (req, res) => {
  const book = books.find(b => b.id === req.params.id);
  if (!book) {
    return res.status(404).json({ error: 'Book not found' });
  }
  res.status(200).json(book.title);
});

```

=> invalid id is provided, books.find() returns undefined. Attempting to access title on undefined throws a type error (cannot read properties of undefined (reading 'title')).

7) `const cors = require('cors');` } correct code
`app.use(cors());`

⇒ CORS (Cross-Origin Resource Sharing) is a browser-enforced security mechanism that blocks client-side scripts from requesting resources across diff. origins

local host: 3000 vs local host = 5000
Portman bypasses CORS because it is a backend HTTP client, not a web browser executing frontend script.